Multi-agent I: Controller, Worker, Reviewer

Plan, do, check under every name since FIPA; roles and personas; result contracts that fail closed; two-stage review; state graphs with a human interrupt

SURVEY | BUILD | GOVERN | EVALUATE

CIDM 6333 · Agentic Systems Design and Development
Survey: FIPA and BDI · CAMEL, MetaGPT, AutoGen, CrewAI · LangGraph supervisor · Claude Code subagents · Build: an implementer–reviewer pair with a fail-closed contract and a state graph (lab: a read-only reviewer in OpenCode)

Two hours, seven moves

The run of show

1	0:02	**Framing**	Plan / do / check under every name; typed claims; verdicts that cite evidence
2	0:10	**Spine**	Why more than one; personas; contracts; two-stage review; state graphs; read-only reviewer; mistakes; worked trace
·	0:55	**Break**	Five minutes
3	1:00	**Teardowns**	FIPA/BDI; CAMEL and MetaGPT; AutoGen and CrewAI; LangGraph supervisor; Claude Code subagents; a live Design Analysis
4	1:30	**Reference design**	gzkit ADR-0.18.0: five stages, four roles, HandoffResult, dispatch records, "credit is never inferred"
5	1:45	**Lens and card**	A strong Sortie 8; Flight Card 8 brief (expansion)
6	1:54	**Lab and Sprint**	Lab (self-guided); the Sprint, with the Chase's Go/No-Go in the repo

Plan, do, check: the recurring division of labor

The roles never change; what changes is whether the check can say no

FIPA: manager and contractor. MetaGPT: architect and engineer. Claude Code: main agent and subagent. Each time, no reviewer whose FAIL stops the work.

Two rules the spine returns to

Rule one. A worker's result is a typed claim; unknown status is rejected.

Rule two. A verdict counts only if it cites evidence and its author could not edit.

Tonight's claim, which Sortie 8 will try to disprove

No implementer output reaches main without a reviewer verdict; FAIL blocks.
Falsifiers:
merged work with no ReviewResult
a malformed result parsed as PASS or DONE
a reviewer claim accepted without its cited evidence
a pass won by loosening the parser or reviewer permissions

Why more than one agent, and what it costs

Three reasons to split the work; one measured price; one published counter-argument

~15×

tokens of a multi-agent run vs. a chat turn (single agent ~4×), Anthropic research system, Jun 2025

90.2%

better than single-agent Opus 4 on Anthropic's internal research eval (Opus 4 lead, Sonnet 4 workers)

80%

of performance variance on that eval explained by token usage alone

Three reasons to split:

Context isolation. A worker returns a result, not its trace.

Independent review. The reviewer did not write the code.

Parallelism. Independent subtasks run at once.

The counter-argument:

Yan (Cognition, Jun 2025): share full traces; actions carry implicit decisions.

This week: one implementer, one reviewer, in sequence. Parallel workers wait for E9, with locks.

Plan / do / check under every name

Read across a row: who decides, who does, who checks, and what carries the result between them

System	Year	Plans	Does	Checks	What carries the result
Contract net (Smith; FIPA)	1980; 1990s	Manager issues a call for proposals	Contractor that won the bid	Manager, on the inform or failure message	A typed message (performative)
BDI agents (Rao & Georgeff)	1995	Deliberation picks intentions from desires	Plan library executes	Reconsideration when beliefs change	Beliefs about the world
CAMEL	2023	AI user gives instructions	AI assistant answers	Nobody; the assistant says <CAMEL_TASK_DONE>	Free-text chat turns
MetaGPT	2023	Product manager, architect	Engineer	QA role; executable feedback	Documents: PRD, design, code
AutoGen 0.2	2023	Assistant agent	User proxy runs the code	Conversation continues until TERMINATE	Chat messages
CrewAI (hierarchical)	2024	Manager agent	Role agents (role, goal, backstory)	Manager reviews; task guardrail returns (bool, value)	Task output, optionally Pydantic
Claude Code / OpenCode	2025–26	Main agent	Subagent in its own window	Whatever the caller does with the result	A final message to the caller
gzkit pipeline	2026	Controller	Implementer	Spec reviewer, then quality reviewer	HandoffResult, ReviewResult

Sources: Smith (1980); Rao & Georgeff (ICMAS-95); Li et al. (2023); Hong et al. (2023); Wu et al. (2023); CrewAI and Claude Code docs checked Sep 2026. gzkit row:

Roles and personas: traits, anti-traits, and the why

A persona is a specification of behavior you can test, not a claim of expertise

What a persona file holds

Role. One job: implementer, spec reviewer, quality reviewer, narrator.

Traits. Observable: 'cites file and line for every finding.'

Anti-traits. 'does not rewrite the code it reviews.' These carry weight.

The why. What breaks if the role fails.

Delete on sight. 'You are a world-class senior engineer.'

reviewer.md (course example, excerpt)

Role: reviewer. You judge one diff against one brief.

Traits:
- Read the brief and the diff before any summary.
- Every finding cites file:line or a test output.
- A requirement with no test is a FAIL finding.

Anti-traits:
- Do not edit, fix, or propose a patch.
- Do not accept 'tests pass' without the output.
- Do not soften a FAIL into CONCERNS to be agreeable.

Why: the controller merges only on PASS. A false PASS puts a defect on main that no later step checks.

Return exactly one ReviewResult JSON object.

Chapter 17 exercise: remove your reviewer's anti-traits and count how its verdicts change. An Exceptional mark this week.

Result contracts: the worker's claim has a type

Two schemas with closed status sets, and a parser that treats anything else as a failure

HandoffResult (implementer → controller)

status: one of DONE · DONE_WITH_CONCERNS · NEEDS_CONTEXT · BLOCKED

Course fields:

task id

files changed

evidence: the test command and its exit code

concerns (required if DONE_WITH_CONCERNS)

question (required if NEEDS_CONTEXT)

NEEDS_CONTEXT and BLOCKED are honest outcomes.

Statuses from gzkit ADR-0.18.0; field names are the course's.

ReviewResult (reviewer → controller)

verdict: one of PASS · FAIL · CONCERNS

findings: a list, each with

location (file:line, test id, or hunk)

what is wrong

severity

Rule: FAIL and CONCERNS need a finding.

PASS lists what was checked.

No citations: rejected as malformed.

Fail-closed parse (Python)

```
OK = {"DONE", "DONE_WITH_CONCERNS",
      "NEEDS_CONTEXT", "BLOCKED"}

def parse_handoff(text):
    try: r = json.loads(text)
    except ValueError:
        return reject("json")
    if r.get("status") not in OK:
        return reject("status")
    if r["status"].startswith("DONE") \
      and not r.get("evidence"):
        return reject("no evidence")
    return r
```

There is no default branch that returns DONE.

Two-stage review: spec first, then quality

Two reviewers with different questions, in a fixed order, each unable to edit

1 → 2 → 3 → 4

Implement

The implementer gets one task, the brief, and the allowed paths. Returns a HandoffResult; a parse reject stops here.

Spec review

Does the change do what the brief asks, inside the allowed paths? Reads brief and diff. FAIL routes back with findings.

Quality review

Is it done well? Tests meaningful, errors handled, no dead code. Runs only after spec PASS.

Controller decides

Merge only on two PASS verdicts that cite evidence. CONCERNS goes to the human. After N failed rounds: BLOCKED to the human.

Order matters: a quality review of the wrong feature gives a confident PASS. Sortie 8 may use one reviewer; two stages is Exceptional.

State graphs: the controller as nodes, edges, and a checkpoint

LangGraph as the worked example; any equivalent that names its states and persists them counts

The graph (LangGraph, Python)

```
g = StateGraph(State)
g.add_node("implement", implement)
g.add_node("review", review)
g.add_node("merge", merge)
g.add_edge(START, "implement")
g.add_edge("implement", "review")
g.add_conditional_edges("review", route,
 {"pass": "merge", "fail": "implement",
  "halt": END})
app = g.compile(
 checkpointer=InMemorySaver())
```

The human interrupt

```
def merge(state):
   ok = interrupt({"verdict":
state["verdict"]})
   ...
```

interrupt() pauses the graph and saves state through the checkpointer. It needs a thread_id in the config. You resume with Command(resume=True); that value becomes the return of interrupt().

This is where index zero sits in the graph.

Supervisor pattern, 2026

LangChain recommends a supervisor built with tool calling: the main agent calls each subagent as a tool. The langgraph-supervisor library (create_supervisor, last release 0.0.31, Nov 2025) is no longer advised.

Without LangGraph: a state machine with named states and a JSON checkpoint after each node.

Declaring a reviewer that cannot edit

Read-only is a permission in config, enforced by the harness; a sentence in the prompt is only a request

OpenCode: .opencode/agents/reviewer.md

```
---
description: Reviews one diff against one brief. Returns one ReviewResult JSON.
Never edits.
mode: subagent
temperature: 0.1
permission:
  edit: deny
  webfetch: deny
  bash:
    "*": deny
    "git diff*": allow
    "git log*": allow
---
(persona body from the previous slide)
```

Invoke with @reviewer in the TUI, or: opencode run --agent reviewer "...". A primary agent can also call it through the task tool.

Claude Code: .claude/agents/reviewer.md

```
---
name: reviewer
description: Reviews one diff against one brief ...
tools: Read, Grep, Glob
model: sonnet
---
```

tools is an allowlist; disallowedTools removes inherited tools. Same doctrine in both: the deny enforces; the anti-trait explains. Test: ask the reviewer to fix the bug.

OpenCode docs (Sep 2026): the tools key is deprecated; use permission. Hidden subagents can still be called through the task tool.

Common mistakes: how a pair of agents becomes one agent with extra steps

Each of these produced a PASS on a defect in someone's system; each has a mechanical fix

Mistake	What happens	Fix	App. C
Reviewer runs in the implementer's session ('now review your work')	Same context, same blind spots; the review agrees with the author	Separate agent, own window; hand it the brief and the diff only	#4
Verdict read by searching the reply for 'PASS'	'This would not PASS without tests' parses as PASS	JSON with an enum; reject everything else	#9
Parser catches the error and defaults to DONE	A crash or a refusal merges	No default branch; reject, log, halt	#3
Reviewer given the implementer's summary instead of the diff	The summary becomes the fact the reviewer checks	Reviewer reads primary sources: diff, test output	#3
Reviewer has edit or bash write	It fixes the code, then passes its own fix	permission edit: deny; bash allowlist	#1
FAIL loops back with no bound	Retries until the reviewer gives in	Max rounds, then BLOCKED to the human	#9

Row four is the E9 incident in miniature: gzkit's 2026-05-24 chain, summaries becoming facts ten levels deep.

Worked trace: one task, three results, one rejection

What the controller's log should look like after a short run (course example)

```
TASK T-03  add a retention rule to the memory layer (brief B-07, allowed: src/memory/, tests/memory/)

[implement#1] -> {"status":"DONE","task":"T-03","files":["src/memory/store.py"],
                  "evidence":{"cmd":"uv run pytest tests/memory -q","exit":0}}
[parse]       ok
[review#1]    -> {"verdict":"FAIL","findings":[{"loc":"tests/memory/","what":"B-07 REQ-2
                  (evict after 30 days) has no test","sev":"high"}],"checked":["diff","B-07"]}
[route]       fail -> implement (round 2 of 3)

[implement#2] -> Sure! I added the test and everything passes now.
[parse]       REJECT not JSON  -> re-dispatch with the schema attached (round 3 of 3)
[implement#3] -> {"status":"DONE","files":["src/memory/store.py","tests/memory/test_retention.py"],
                  "evidence":{"cmd":"uv run pytest tests/memory -q","exit":0}}
[parse]       ok
[review#2]    -> {"verdict":"PASS","findings":[],"checked":["diff","B-07 REQ-1..3",
                  "test_retention.py::test_evicts_after_30_days"]}
[interrupt]   checkpoint 7f3a  waiting for human
[resume]      approved by <student>  -> merge 1c9e2d4

DISPATCH RECORDS  implement x3 (1 rejected) · review x2 · human x1
CHASE READS       this log, both ReviewResults, git show 1c9e2d4 --stat; re-runs the test command
```

Break · 0:55–1:00

Five minutes

In your system, which component could judge the implementer's work today, and what could it edit while doing so?

Teardown: FIPA and BDI, the pre-LLM baseline

Typed messages, commitments, and reconsideration, thirty years before the chat transcript

FIPA and the contract net

Contract net (Smith, 1980): call for proposals, bids, winner, result or failure.

FIPA (1996; IEEE from 2005): every message carries a performative.

Mechanism. Message type is claim type.

Design choice. Declared vocabulary.

Cost. Ontologies, heavy tooling.

Failure mode. Was the inform true? Unsaid (#3).

BDI: beliefs, desires, intentions

Source. Rao & Georgeff, 'BDI Agents: From Theory to Practice' (ICMAS-95).

Mechanism. Beliefs, desires, committed intentions; reconsidered when beliefs change.

Carried forward. Goal decomposition; intent preservation; halt as 'intention achieved or dropped.'

LLM agents dropped. Inspectable belief state; a transcript is not a belief base.

Read FIPA before AutoGen and CrewAI (Ch. 4): performatives became status enums; contract net became supervisor; reconsideration became replanning.

Teardown: CAMEL and MetaGPT, roles in 2023

Role play by prompt, then roles connected by documents

CAMEL (Li et al., NeurIPS 2023)

Mechanism. An AI user and an AI assistant, set up by 'inception prompting', no human.

Design choice. Study agent societies through their conversations.

Failure. Four named: role flipping, repeated instructions, flake replies, infinite loops.

Halt: the assistant writes <CAMEL_TASK_DONE>: completion by assertion (#3, #9).

arXiv 2303.17760

MetaGPT (Hong et al., 2023)

Mechanism. 'Code = SOP(Team)': product manager, architect, project manager, engineer, QA. Structured documents (PRD, design, code), not dialogue; executable feedback.

Design choice. An assembly line against cascading hallucination.

Failure. 85.9% / 87.7% Pass@1 on HumanEval / MBPP. A wrong PRD flows downstream intact (#4).

arXiv 2308.00352

What changed between them

CAMEL connected roles with chat; MetaGPT with shaped documents.

From 'the agents talked' to 'the agents handed over an artifact.'

This week's contract goes one further: a closed status set; anything else rejected.

Neither had a blocking reviewer or a human in the path.

Teardown: AutoGen and CrewAI, roles as configuration

Conversation as the coordination mechanism, then a manager with a guardrail

	AutoGen (Wu et al., 2023) → Microsoft Agent Framework	CrewAI
Mechanism	Conversable agents: assistant writes code, user proxy runs it, manager picks next speaker. Halt: TERMINATE.	Role, goal, backstory per agent. Hierarchical: a manager (manager_llm or manager_agent) plans, delegates, reviews.
Design choice	Coordination is a conversation; programming it means programming who talks next.	Coordination is an org chart; the manager is the reviewer.
Check	The code actually runs in the user proxy: an observation, not a claim.	Task guardrail returns (bool, value); on False, retry, default 3 times. output_pydantic types the result.
Status, 2026	AutoGen: maintenance mode. Microsoft Agent Framework 1.0 (Apr 3, 2026): sequential, concurrent, handoff, group chat, Magentic-One; checkpointing; human approvals.	Crews plus Flows; hierarchical process documented; no consensual process in current docs.
Failure mode	Chat ends when someone says TERMINATE (#9); speaker selection is another model call.	Manager and workers often share one model; retry-until-pass guardrails can be metagamed (#9).

Sources checked Sep 2026: github.com/microsoft/autogen; devblogs.microsoft.com/agent-framework (1.0 post); docs.crewai.com (processes, tasks).

Teardown: the LangGraph supervisor

A graph with a hub node; the question is what passes back through the hub

Mechanism

A supervisor node picks the next worker; workers return to it; edges and state are explicit; a checkpointer persists state at every step.

The library version (create_supervisor) chose output_mode: full_history (every worker message) or last_message (only the final reply).

Design choice and cost

Choice: the controller is code you can read; the supervisor is one node in it.

Cost: every routing decision is a model call at the hub. full_history fills the supervisor's window; last_message hides the worker's trace. Cognition's argument, set as a parameter.

Failure mode and the 2026 shift

Failure: with last_message the supervisor sees only the worker's summary and treats it as the result (#3, summary laundering).

2026: LangChain recommends a supervisor built with tool calling. The docs name five patterns: subagents, handoffs, skills, router, custom workflow.

Teardown: Claude Code subagents (and OpenCode's)

Harness-native workers: a Markdown file, a tool allowlist, a clean window, and a message back

Subagents (Claude Code docs, Sep 2026)

Mechanism. Markdown file with frontmatter (name, description, tools, model, permissionMode, maxTurns, hooks, isolation). Own window; result returns to caller.

Nesting. Three levels deep by default.

Built-ins. Explore and Plan (read-only), general-purpose.

OpenCode. Same shape: primary agents (Build, Plan); subagents (General, Explore, Scout).

Agent teams (experimental) and what they teach

Mechanism. Lead, teammates, shared task list, mailboxes. Off by default (CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1).

Docs: 'A teammate can't approve a permission prompt or supply consent on your behalf.' The lead approves a plan 'without the lead reviewing it.'

Reading. Authority does not cross (E5); auto-approval cannot say no.

Failure. Caller trusts the message (#3).

Design choice: context isolation first, review second. The harness gives the worker; you supply the contract and the verdict.

Design Analysis, live: Claude Code subagents (2025–26)

Worked on the board; the class fills the Coordination and Evaluation rows before the answers appear

Component	Answer (from the docs, Sep 2026)
Foundation	Claude models; per-subagent model: sonnet, opus, haiku, a full id, or inherit
Perception	Delegation message, CLAUDE.md, git status snapshot, preloaded skills; not the parent's conversation
Planning & reasoning	Parent delegates on the description; the subagent plans in its own loop, bounded by maxTurns
Tools & orchestration	tools allowlist or disallowedTools; the Agent tool spawns workers; permissions can deny Agent(name)
Memory & context	Own window; result summarized back to the caller; optional memory scope; resumable by message
Coordination	Hub and spoke: the caller manages all work; teams add a task list and mailboxes
Evaluation & feedback	No verdict schema; the caller reads a message. Hooks (PreToolUse, PostToolUse, Stop) can hold a check
Governance & human interface	permissionMode per subagent; teammates cannot give consent for the human; @-mention forces a delegation
Runtime & operations	Foreground or background; worktree isolation; token cost rises with each worker's window

Judgment loop: Decision: isolate context per worker · Privileges: clean windows, cheap parent context · Cost: tokens, lost trace · Failure: summary accepted as fact · Preservation: none by default; your contract

Reference design: gzkit's controller–worker pipeline (ADR-0.18.0)

Five stages; the controller never implements; every dispatch leaves a record

1 → 2 → 3 → 4 → 5

1 · Load context

The controller reads the brief (OBPI), its requirements, and its allowed paths. Nothing is dispatched without a brief.

2 · Implement

One dispatch per task to the implementer; HandoffResult back; then spec reviewer, then quality reviewer, each returning ReviewResult.

3 · Verify

The controller runs tests, lint, and the brief's verification commands itself. A worker's 'tests pass' is not this stage.

4 · Present evidence

Paths, commands, and outputs go to the human, not conclusions. (E9: Step 4b adds adversarial validation.)

5 · Sync

Ledger events, brief status, and dispatch records written. The human attests; the pipeline does not.

Sources: docs/user/concepts/subagent-pipeline.md; src/gzkit/roles.py; pipeline_dispatch.py; .gzkit/personas/.

Reference design: HandoffResult, dispatch records, 'credit is never inferred'

A role that was not dispatched did not happen, whatever the transcript says

The contract and the record

HandoffResult. DONE · DONE_WITH_CONCERNS · NEEDS_CONTEXT · BLOCKED.

ReviewResult. PASS · FAIL · CONCERNS with findings. Parsing is fail-closed.

Dispatch records. Every dispatch of every role writes a record.

'Credit is never inferred.' No dispatch record for the spec reviewer: spec review did not happen.

The honesty flag

One agent may do everything, declared: --single-driver --reason "...".

Declared: graded as single-driver. Undeclared, presented as reviewed: falsifier three.

Reviewer could not run (Big Pickle unavailable, Go limit reached): ABORT, not PASS.

Reference design: why the rule exists

Two dated episodes from Appendix G; both return in E9

> *A subagent's claim is not evidence. A presence check answers 'is something armed', never 'did the procedure run.'*

2026-05-24: ten-deep chain; summaries of summaries became 'facts'. Hence the DO IT RIGHT rules.

2026-08-21: undispatched Stage 2; a stale marker licensed roughly 350 lines, no dispatch, no review. Hence dispatch records.

Evaluation lens: a strong Sortie 8

The Chase reads the dispatch log, the ReviewResults, the parser's rejections, and git log; transcripts are not evidence

Weak

- Reviewer is the implementer's session told 'review your work'
- Verdict found by searching the reply for 'PASS'
- Parser catches errors and continues as DONE
- Reviewer can edit or run arbitrary bash
- Persona says 'expert engineer'; no anti-traits
- For-loop controller; no checkpoint; no human before merge

Sound

- Separate reviewer, own window, read-only permission in config
- Closed-enum HandoffResult and ReviewResult schemas; malformed input rejected
- FAIL blocks merge; bounded rounds, then BLOCKED
- Findings cite file:line, test id, or output
- Graph or state machine; checkpoint; interrupt before merge
- Dispatch record per role per task

Exceptional

- Two-stage review with separate spec and quality personas
- Reviewer on a different model; both ids logged
- Anti-traits removed and verdicts re-counted (Ch. 17 exercise)
- A planted defect the reviewer missed, reported as a squawk
- Chase re-runs the evidence commands; same exit codes

Flight Card 8: a reviewer that can say no

Expansion · depends on Sortie 7 · Go/No-Go by the Chase, in the repo

```
ENVELOPE            expansion
DESIGN CLAIM        No implementer output reaches main without a reviewer verdict, and a FAIL blocks.
SYSTEMS UNDER TEST  controller (graph or state machine) · implementer persona · reviewer persona (read-only)
                    · HandoffResult + ReviewResult schemas · fail-closed parser · merge gate · checkpoint
                    · ledger
SUBSTRATE           repo + commit · graph library + version · implementer model id · reviewer model id
DEPENDS ON          Sortie 7 PASS (memory layer), or its re-fly on record

ENTRY CONDITIONS    clean tree · ADR for the pair (roles, contract, graph) committed · Chase GO in the card

TEST POINTS
  1  clean task          -> HandoffResult DONE, parsed -> ReviewResult PASS citing diff -> interrupt, merge
  2  planted defect      -> ReviewResult FAIL with a located finding -> routed to implement -> no merge
  3  malformed result    -> parser REJECT logged (not JSON / unknown status / DONE without evidence)
                            -> no review dispatched, no merge
  4  'fix it yourself'   -> reviewer's edit denied by the harness; diff unchanged; verdict still returned
  5  resume after halt   -> checkpoint id logged; human approval recorded; exactly one merge commit

INSTRUMENTATION     dispatch log (one record per role per task) · 2+ ReviewResults · 1+ REJECT lines
                    · checkpoint ids · git log · ledger events

PRE-REGISTERED FALSIFIERS (any ONE => FAIL)
  - merged work with no ReviewResult on record for that task
  - a malformed result parsed as PASS or DONE
  - a reviewer claim accepted without the evidence it cites
  - passing only by loosening the parser, granting the reviewer edit, or editing the log

ABORT (halt-not-fail)  model route unavailable for either role; usage limit reached mid-sortie
CHASE               reads dispatch log, ReviewResults, REJECT lines, git log; re-runs one evidence command

SPRINT              Sortie 8 log · pair + contract + graph · Design Analysis · AI_LOG · brief acknowledged
```

1:54–1:57

Lab tonight: a read-only reviewer in OpenCode

Self-guided · handout CIDM6333_Lab_E8_Subagents.md · Big Pickle or a Go model · no installs

Thirty minutes, five steps

0–4 Scratch branch; copy the handout files.

4–10 Write .opencode/agents/reviewer.md: anti-traits, read-only permissions.

10–17 @reviewer the diff; parse the reply with parse_review.py.

17–23 Check each file:line; run the test yourself.

23–27 Say 'just fix it'; record the refusal. Three malformed replies.

27–30 Commit; worksheet.

What you commit

evolution-08/lab/
reviewer.md
review-1.json (raw reply) and the parser's verdict
worksheet.md: verdict checked, the refusal, three REJECTs, model id
A ledger event for the commit.
Small models help. Prose instead of JSON: a REJECT you record. A nonexistent cited line: a refuted claim.

The Sprint

Homework brief: CIDM6333_HW_E8_MultiAgent1.md · Go/No-Go and Chase review happen in writing, in the repo

Hand in

- Sortie 8 flight log (card, evidence, verdict, debrief)
- The pair: personas, both schemas, fail-closed parser, checkpointed graph with interrupt, ADR
- Design Analysis (one multi-agent system)
- AI_LOG entry
- Final System brief acknowledged

Asynchronous Go/No-Go

Card committed to evolution-08/flight-card.md.

Your Chase signs GO or NO-GO under CHASE, one line of reason, as a commit.

No sortie flies before GO is in the log. NO-GO: amend and ask again.

Chase review after

Flight log committed.

The Chase reads only the evidence slice named on the card.

One evidence command re-run; PASS or FAIL with one line of reason.

The Chase does not read your narrative first.

A worker's result is a claim with a type. A verdict counts only if its author could not edit the work.

Build the reviewer that can say no, and the controller that listens. Commit the card; your Chase signs. Lab: declare the reviewer and check what it tells you.